

AI VIET NAM  
@aivietnam.edu.vn

Conquer

Đăng nhập

Đăng ký



#conq999

🕒 26 phút đọc 👁 419 lượt xem ❤️ 0 thích 💬 0 bình luận

# 👁 MonitorAI - Thiên La Địa Vồng: Prometheus, Loki và Grafana Giám Sát LLM



Đàm Nguyễn Khánh

🎓 Tác giả chính

Xuất bản: 26/11/2025

Cập nhật: 23/12/2025



## Tóm tắt

Khi chạy nhiều LLM models cùng lúc, ta thường gặp khó khăn: không biết process nào đang chạy, CPU/GPU đang dùng bao nhiêu, hay logs nằm ở đâu. **MonitorAI** giải quyết vấn đề này bằng cách tự động phát hiện các LLM processes, thu thập metrics CPU/Memory/GPU, và tập trung logs vào một dashboard đẹp mắt. Bài viết này sẽ dẫn dắt ta từng bước để hiểu cách hệ thống hoạt động và cách sử dụng nó trong thực tế.

*"Từ bài toán giám sát LLM processes với hàng trăm metrics và logs — cùng tìm hiểu cách MonitorAI xây dựng hệ thống monitoring toàn diện với Grafana, Prometheus, Loki và Tempo."*



Bài viết được biên soạn bởi nhóm CONQ999

# 1. Giới thiệu: Bài toán thực tế

## 1.1 Thách thức khi giám sát LLM Processes

Hãy tưởng tượng ta đang chạy 3-4 LLM models cùng lúc trên máy tính. Mỗi model có thể là GPT-2, LLaMA, hay Mistral. Làm sao ta biết được:

- Process nào đang chạy model nào?
- CPU và Memory đang dùng bao nhiêu?
- GPU có đang bị quá tải không?
- Logs nằm ở đâu khi có lỗi?

Đây chính là những câu hỏi mà **MonitorAI** giúp ta trả lời. Hệ thống tự động phát hiện các LLM processes, thu thập metrics, và hiển thị mọi thứ trên một dashboard dễ đọc.

💡 **Dự án mã nguồn mở:** Toàn bộ code và cấu hình của hệ thống này đã được công khai trên GitHub tại <https://github.com/D9Dre4mer/MonitorAI>.

## 1.2 Các LLM Frameworks được hỗ trợ

MonitorAI có thể tự động phát hiện nhiều frameworks phổ biến:

- **Hugging Face Transformers** - Framework phổ biến nhất cho LLM
- **llama.cpp** - Inference engine tối ưu cho llama models

- **vLLM** - High-throughput LLM serving engine
- **TensorRT** - NVIDIA's inference optimization framework
- **ONNX Runtime** - Cross-platform inference engine
- **PyTorch** - Deep learning framework
- **TensorFlow** - Google's machine learning platform

Hệ thống cũng tự động trích xuất tên model từ command line. Ví dụ, nếu ta chạy `python run-model.py --model-name gpt2`, hệ thống sẽ biết model đang chạy là `gpt2`.

## 1.3 Thách thức kỹ thuật

Trên thực tế, việc giám sát LLM processes không đơn giản. Ta gặp nhiều thách thức:

**Thứ nhất**, mỗi framework có cách detect khác nhau. Transformers dùng `from_pretrained()`, còn llama.cpp lại có pattern riêng. Ta cần pattern matching thông minh để phát hiện tất cả.

**Thứ hai**, trên Windows, `nvidia-smi` không thể query GPU memory per process một cách chính xác. Ta phải dùng cách khác: để process tự ghi GPU memory vào file JSON, rồi đọc file đó. (Trên Linux, `nvidia-smi` có thể query chính xác hơn, nhưng file-based approach vẫn là lựa chọn tốt để đảm bảo tính nhất quán.)

**Thứ ba**, logs từ nhiều processes nằm rải rác. Ta cần tập trung chúng lại để dễ tìm kiếm.

**Cuối cùng**, metrics cần cập nhật nhanh nhưng không được quá tải hệ thống. Ta chọn 10-15 giây là khoảng thời gian hợp lý.

## 1.4 Giải pháp MonitorAI

MonitorAI giải quyết các thách thức trên bằng cách:

- **Tự động phát hiện**: Dùng regex patterns để tìm LLM processes trong command line
- **File-based GPU exposure**: Processes ghi GPU memory vào JSON files, LLM Monitor đọc và expose metrics
- **Prometheus metrics**: Dùng format chuẩn, dễ tích hợp với nhiều tools
- **Loki logs**: Tập trung logs với LogQL để query dễ dàng

- **Forest Green theme:** Dashboard đẹp, dễ đọc, thống nhất màu sắc
- **Auto cleanup:** Tự động xóa metrics sau 10 giây nếu process đã dừng

💡 **Fun fact về Prometheus:** Trong thần thoại Hy Lạp, Prometheus là vị thần Titan đã đánh cắp lửa từ Olympus và mang đến cho loài người, tượng trưng cho sự khai sáng và tri thức. Công cụ Prometheus cũng vậy - nó "mang lửa" giám sát hệ thống đến cho ta, giúp ta "thấy ánh sáng" trong việc theo dõi metrics!

💡 **Fun fact về Loki:** Loki là vị thần lừa đảo trong thần thoại Bắc Âu, nổi tiếng với khả năng biến hình và tinh quái. Công cụ Loki cũng "biến hóa" logs của ta một cách linh hoạt, giúp ta "bắt quả tang" những vấn đề ẩn giấu trong hệ thống!

## 1.5 Tại sao chọn Observability Stack?

Ta chọn stack Grafana + Prometheus + Loki + Tempo vì đây là những công cụ mạnh mẽ và phổ biến:

💡 **Fun fact về Grafana:** Tên "Grafana" xuất phát từ chữ "graph" (đồ thị) và hậu tố "-ana", gợi nhớ đến "Nirvana" (niết bàn). Có lẽ nhóm phát triển muốn ta đạt đến "cõi niết bàn" của dữ liệu khi xem dashboard đẹp mắt này! 😊

- **Standard protocols:** Prometheus metrics, LogQL, OTLP traces - tất cả đều là chuẩn công nghiệp
- **Scalable:** Có thể mở rộng cho nhiều hosts và services
- **Rich ecosystem:** Nhiều exporters và integrations sẵn có
- **Beautiful dashboards:** Grafana có nhiều visualization options
- **Open source:** Miễn phí, cộng đồng lớn
- **Production-ready:** Được sử dụng rộng rãi trong production

## 2. Pipeline Hệ Thống MonitorAI

### 2.1 Sơ đồ tổng quan Pipeline

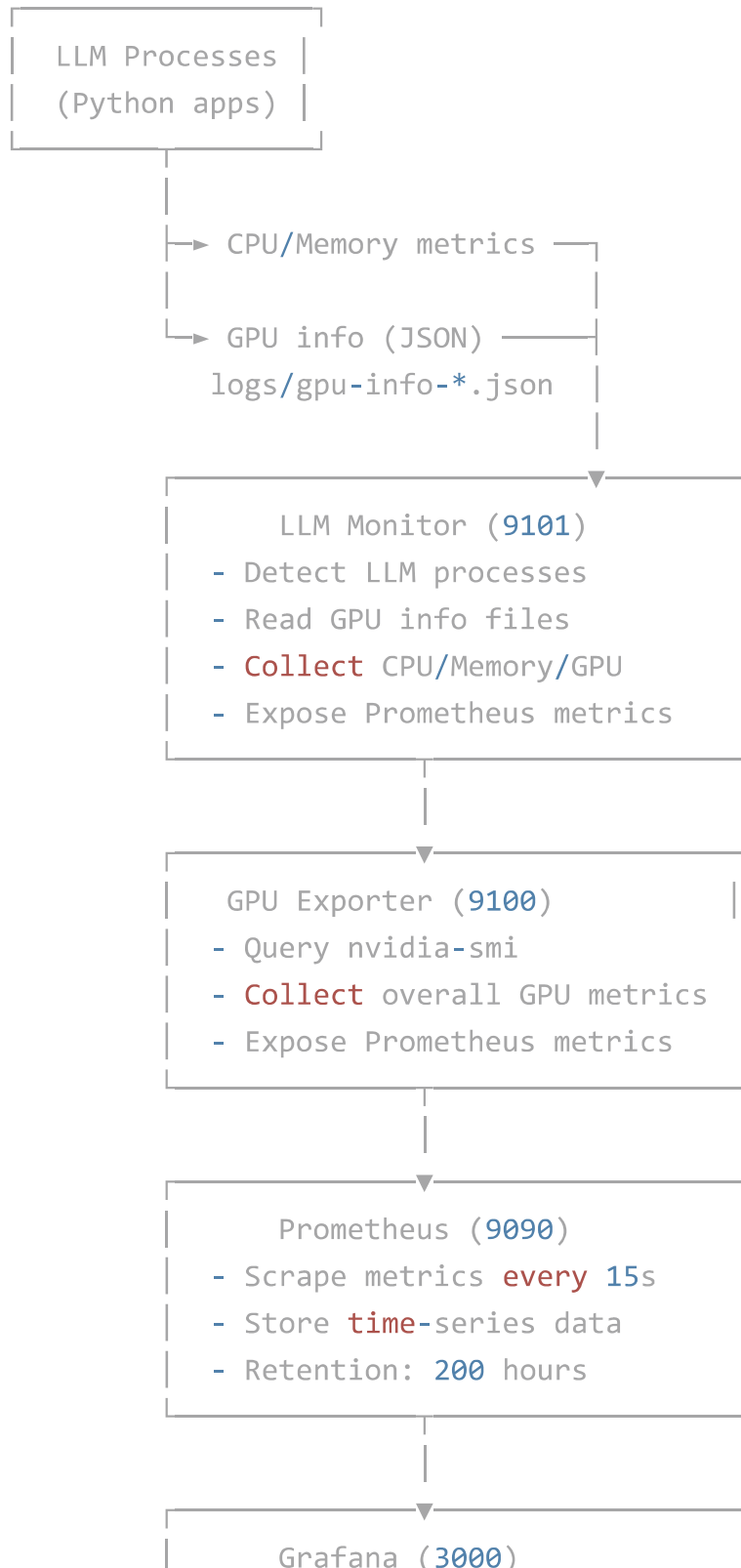
Hệ thống MonitorAI hoạt động theo 3 pipeline chính:

1. **Metrics Pipeline** - Thu thập CPU, Memory, GPU metrics
2. **Logs Pipeline** - Tập trung logs từ nhiều processes

### 3. Tracing Pipeline - Distributed tracing (tùy chọn, tương lai)

Hãy xem từng pipeline hoạt động như thế nào.

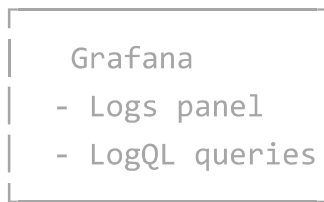
#### Pipeline Metrics (CPU, Memory, GPU):



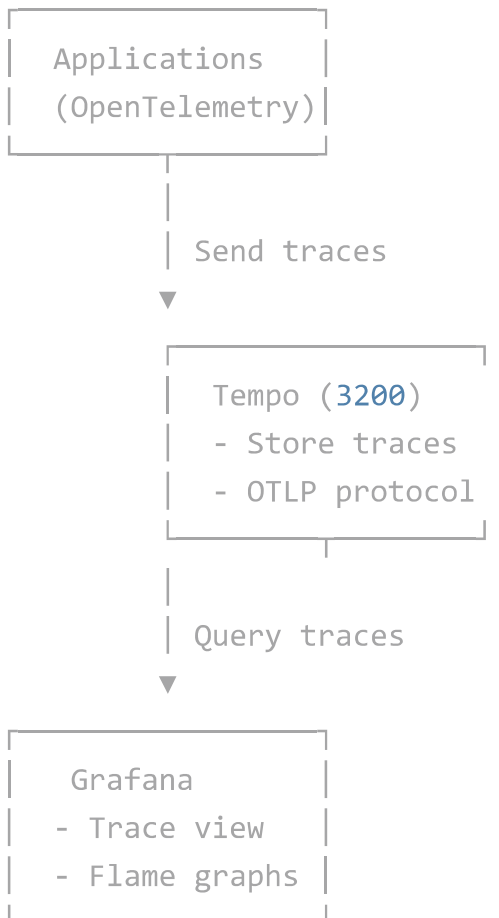
- Query Prometheus via PromQL
- Visualize **in** dashboards
- Forest Green theme

## Pipeline Logs:





### Pipeline Tracing (Optional - Future):



💡 **Fun fact về Tempo:** "Tempo" trong tiếng Ý có nghĩa là "nhịp độ" trong âm nhạc. Công cụ Tempo cũng vậy - nó theo dõi "nhịp độ" của traces trong hệ thống, giúp ta hiểu được timing và flow của requests qua các services!

## 2.2 6 Bước chi tiết theo Pipeline

### 2.2.1 Bước 1: Phát hiện LLM Processes

Hệ thống bắt đầu bằng việc quét tất cả Python processes đang chạy. Giống như một "radar" quét toàn bộ hệ thống để tìm các LLM processes.

Ta sử dụng thư viện `psutil` để lấy danh sách tất cả processes. Sau đó, ta kiểm tra command line của mỗi process xem có chứa từ khóa của LLM frameworks không.

### Cách hoạt động:

Ta định nghĩa các patterns cho từng framework. Ví dụ, với Transformers, ta tìm các từ khóa như `transformers`, `huggingface`, hay `from_pretrained`. Khi tìm thấy, ta biết process đó đang chạy LLM.

```
LLM_PATTERNS = {
    'transformers': [
        r'transformers',
        r'huggingface',
        r'\.from_pretrained',
        r'pipeline\(.model',
    ],
    'llama.cpp': [
        r'llama',
        r'llama\.cpp',
        r'gguf',
    ],
    'vllm': [
        r'vllm',
        r'vllm\.engine',
    ],
    # ... các patterns khác
}

def detect_llm_processes():
    processes = []
    for proc in psutil.process_iter(['pid', 'name', 'cmdline']):
        cmdline = ' '.join(proc.info['cmdline'] or [])
        for framework, patterns in LLM_PATTERNS.items():
            if any(re.search(pattern, cmdline, re.IGNORECASE)
                    for pattern in patterns):
                processes.append({
```



```

        'pid': proc.info['pid'],
        'framework': framework,
        'model_name': extract_model_name(cmdline)
    })
    return processes

```

### Kết quả thực tế:

Sau khi quét, ta có thể phát hiện được:

- **Process 15176:** `transformers` framework, model `gpt2`
- **Process 22716:** `transformers` framework, model `DialogPT-small`

## 2.2.2 Bước 2: Thu thập Metrics

Sau khi phát hiện processes, ta cần thu thập metrics. Ta quan tâm đến 4 loại metrics chính: CPU usage, Memory usage, GPU memory, và GPU utilization.

### CPU và Memory:

Ta dùng `psutil` để lấy CPU percentage và memory usage (RSS) cho từng process. Đây là cách đơn giản và chính xác.

### GPU metrics:

Đây là phần phức tạp hơn. Trên Windows, `nvidia-smi` không thể query GPU memory per process chính xác. Vì vậy, ta dùng cách khác: process tự ghi GPU memory vào file JSON, rồi ta đọc file đó.

```

def collect_process_metrics(pid):
    proc = psutil.Process(pid)

    # CPU và Memory
    cpu_percent = proc.cpu_percent(interval=1.0)
    memory_bytes = proc.memory_info().rss

    # GPU metrics từ file JSON
    gpu_info_file = Path(f'logs/gpu-info-{pid}.json')
    if gpu_info_file.exists():
        with open(gpu_info_file) as f:

```

```

gpu_info = json.load(f)
gpu_memory =
gpu_info.get('gpu_memory_allocated_bytes', 0)
gpu_util = gpu_info.get('gpu_utilization', 0)

return {
    'cpu_percent': cpu_percent,
    'memory_bytes': memory_bytes,
    'gpu_memory_bytes': gpu_memory,
    'gpu_utilization': gpu_util
}

```

### Kết quả thực tế:

Sau khi thu thập, ta có thể thấy:

- **Process 15176** (gpt2): Memory ~254 MB, GPU Memory ~21-26 GiB
- **Process 22716** (DialogPT-small): Memory ~307 MB, GPU Memory 0 GiB (không sử dụng GPU)

### 2.2.3 Bước 3: Expose Prometheus Metrics

Sau khi thu thập metrics, ta cần expose chúng theo format Prometheus.

Prometheus sẽ scrape metrics từ endpoint `/metrics` của ta.

#### Cách hoạt động:

Ta dùng thư viện `prometheus_client` để tạo các Gauge metrics. Mỗi metric có labels để ta có thể filter và group dễ dàng. Ví dụ, `llm_process_cpu_percent` có labels `pid`, `name`, `model_name`, `framework`.

```

from prometheus_client import Gauge, start_http_server

# Define metrics
llm_process_cpu_percent = Gauge(
    'llm_process_cpu_percent',
    'CPU usage percentage per LLM process',
    ['pid', 'name', 'llm_type', 'model_name', 'framework']
)

```

```

llm_process_memory_bytes = Gauge(
    'llm_process_memory_bytes',
    'Memory usage in bytes per LLM process',
    ['pid', 'name', 'llm_type', 'model_name', 'framework']
)

# Start HTTP server
start_http_server(9101)

# Update metrics
for process in detected_processes:
    metrics = collect_process_metrics(process['pid'])
    llm_process_cpu_percent.labels(
        pid=process['pid'],
        name=process['name'],
        llm_type='llm',
        model_name=process['model_name'],
        framework=process['framework']
    ).set(metrics['cpu_percent'])

```

### Kết quả:

Metrics được expose tại <http://localhost:9101/metrics> với format Prometheus standard. Prometheus có thể scrape và lưu trữ chúng.

## 2.2.4 Bước 4: Scrape và Lưu trữ

Prometheus tự động scrape metrics từ LLM Monitor và GPU Exporter mỗi 15 giây. Ta chọn 15 giây vì đủ nhanh để capture changes nhưng không quá tải hệ thống.

💡 **Fun fact:** Như vị thần Prometheus trong thần thoại đã "đánh cắp lửa" từ Olympus, công cụ Prometheus của ta cũng "đánh cắp" metrics từ các services mỗi 15 giây, mang lại "ánh sáng" cho việc giám sát hệ thống!

### Cấu hình:

Ta cấu hình Prometheus để scrape từ 2 targets: LLM Monitor (port 9101) và GPU Exporter (port 9100). Prometheus sẽ lưu trữ metrics với retention 200 giờ - đủ để phân tích trends.

```
# prometheus.yml
scrape_configs:
  - job_name: 'llm-monitor'
    scrape_interval: 15s
    static_configs:
      - targets: ['host.docker.internal:9101']

  - job_name: 'gpu-exporter'
    scrape_interval: 15s
    static_configs:
      - targets: ['host.docker.internal:9100']
```

### Kết quả:

Prometheus lưu trữ metrics với labels đầy đủ. Ta có thể query bằng PromQL:

- `llm_process_cpu_percent{framework="transformers"}` - CPU usage của tất cả Transformers processes
- `llm_process_gpu_memory_bytes{model_name="gpt2"}` - GPU memory của model gpt2

## 2.2.5 Bước 5: Tập trung Logs

Logs từ LLM processes được ghi vào file `logs/llm-model.log`. Promtail đọc file này và ship đến Loki để tập trung.

💡 **Fun fact về Promtail:** Tên này là sự kết hợp của "Prometheus" + "tail" (lệnh Unix để theo dõi log files). Nó hoạt động như một "đuôi" theo dõi log files và ship đến Loki - giống như một con mèo đuôi dài luôn theo dõi mọi thứ! 🐱

### Cách hoạt động:

Promtail hoạt động như một "log shipper". Nó đọc log file, parse và thêm labels, rồi push đến Loki. Loki lưu trữ logs với indexing theo labels để query nhanh.

```
# promtail-config.yml
scrape_configs:
  - job_name: llm-model
    static_configs:
```

```
- targets:
  - localhost
labels:
  job: llm-model
  __path__: /logs/llm-model.log
```

### Kết quả:

Logs được tập trung trong Loki. Ta có thể query bằng LogQL:

- `{job="llm-model"} |= "error"` - Tìm tất cả error logs
- `{job="llm-model"} | json | model_name="gpt2"` - Filter logs theo model name

## 2.2.6 Bước 6: Visualization

Cuối cùng, Grafana query metrics từ Prometheus và logs từ Loki, rồi hiển thị trên dashboard với Forest Green theme.

💡 **Fun fact:** Như tên gọi gợi nhớ đến "Nirvana" (niết bàn), Grafana đưa ta đến "cõi niết bàn" của dữ liệu - nơi mọi metrics và logs được trực quan hóa một cách đẹp mắt, giúp ta đạt được sự "giác ngộ" về trạng thái hệ thống! 😊

### Cách hoạt động:

Grafana kết nối đến Prometheus và Loki qua datasources. Ta dùng PromQL để query metrics và LogQL để query logs. Dashboard tự động refresh mỗi 15 giây để cập nhật dữ liệu mới nhất.

```
{
  "panels": [
    {
      "title": "KPI-CPU Usage",
      "targets": [{
        "expr":
"llm_process_cpu_percent{framework=\"transformers\"}"
      }],
      "type": "stat",
      "fieldConfig": {
        "defaults": {
          "color": {"fixedColor": "#27AE60"},

```

```
"thresholds": {
  "steps": [
    {"color": "green", "value": 0},
    {"color": "yellow", "value": 70},
    {"color": "red", "value": 90}
  ]
}
```

Kết quả:

Dashboard hiển thị đầy đủ metrics với Forest Green theme, dễ đọc và phân tích. Ta có thể thấy CPU, Memory, GPU usage của từng process một cách trực quan.

### 3. Áp dụng MonitorAI: Chi tiết kỹ thuật

#### 3.1 Kiến trúc hệ thống

Hệ thống MonitorAI gồm 7 components chính, mỗi component có vai trò riêng:

| Component    | Port | Chức năng                                      |
|--------------|------|------------------------------------------------|
| LLM Monitor  | 9101 | Phát hiện và thu thập metrics từ LLM processes |
| GPU Exporter | 9100 | Thu thập overall GPU metrics từ nvidia-smi     |
| Prometheus   | 9090 | Scrape và lưu trữ metrics                      |
| Loki         | 3100 | Lưu trữ logs                                   |
| Tempo        | 3200 | Lưu trữ traces (future)                        |
| Grafana      | 3000 | Visualization dashboard                        |

| Component | Port | Chức năng          |
|-----------|------|--------------------|
| Promtail  | -    | Ship logs đến Loki |

### Data Flow:

Dữ liệu chảy qua hệ thống theo 3 luồng:

- **Metrics:** LLM Processes → GPU JSON files → LLM Monitor → Prometheus → Grafana
- **Logs:** LLM Processes → Log files → Promtail → Loki → Grafana
- **GPU Overall:** nvidia-smi → GPU Exporter → Prometheus → Grafana

## 3.2 GPU Monitoring trên Windows và Linux

### 3.2.1 Trên Windows

#### Vấn đề:

Trên Windows, `nvidia-smi` không thể query GPU memory per process một cách chính xác. Đây là hạn chế của Windows, không phải của `nvidia-smi`.

#### Giải pháp:

Ta dùng cơ chế **file-based exposure**. Process tự ghi GPU memory vào file JSON, rồi LLM Monitor đọc file đó.

#### Bước 1: Process tự expose GPU memory

Khi chạy LLM model, ta ghi GPU memory vào file `logs/gpu-info-{PID}.json`. File này được cập nhật mỗi lần inference (mỗi 10 giây).

```
# Trong run-llm-model-gpu.py
def save_gpu_info(pid):
    gpu_info = {
        'pid': pid,
        'gpu_memory_allocated_bytes':
            torch.cuda.memory_allocated(0),
        'gpu_memory_reserved_bytes':
            torch.cuda.memory_reserved(0),
```

```

'gpu_utilization': get_gpu_utilization(),
'gpu_index': 0,
'timestamp': datetime.now().isoformat()
}

gpu_info_file = Path(f'logs/gpu-info-{pid}.json')
with open(gpu_info_file, 'w') as f:
    json.dump(gpu_info, f)

```

## Bước 2: LLM Monitor đọc file JSON

LLM Monitor đọc tất cả file `logs/gpu-info-*.json` mỗi 10 giây. Nếu process không còn tồn tại, ta tự động xóa file.

```

# Trong llm_monitor.py
def read_gpu_info_files():
    gpu_info_files = Path('logs').glob('gpu-info-*.json')
    for file in gpu_info_files:
        pid = int(file.stem.split('-')[-1])
        if not psutil.pid_exists(pid):
            file.unlink() # Xóa file nếu process không còn tồn
            tại
            continue

        with open(file) as f:
            gpu_info = json.load(f)
            # Expose metrics từ gpu_info

```

### Lợi ích:

Cách này cho ta GPU memory chính xác từ PyTorch, không phụ thuộc vào `nvidia-smi`. Độ chính xác cao hơn nhiều.

## 3.2.2 Trên Linux

### Khác biệt:



Trên Linux, `nvidia-smi` có thể query GPU memory per process một cách chính xác hơn nhiều so với Windows. Ta có thể dùng lệnh:

```
nvidia-smi --query-compute-apps=pid,process_name,used_memory --format=csv
```

Lệnh này sẽ trả về danh sách các processes đang sử dụng GPU cùng với memory usage của từng process.

### Giải pháp:

Trên Linux, ta có 2 lựa chọn:

#### Option 1: Dùng `nvidia-smi` trực tiếp (Đơn giản hơn)

LLM Monitor có thể query `nvidia-smi` trực tiếp để lấy GPU memory per process. Không cần file-based exposure.

```
# Trên Linux, có thể dùng nvidia-smi trực tiếp
import subprocess

def get_gpu_memory_per_process():
    result = subprocess.run(
        ['nvidia-smi', '--query-compute-apps=pid,used_memory',
        '--format=csv,noheader,nounits'],
        capture_output=True,
        text=True
    )
    # Parse kết quả và trả về dict {pid: memory_bytes}
    return parse_nvidia_smi_output(result.stdout)
```

#### Option 2: Vẫn dùng file-based exposure (Nhất quán với Windows)

Nếu ta muốn code chạy được trên cả Windows và Linux mà không cần thay đổi, ta vẫn có thể dùng file-based exposure. Cách này đảm bảo tính nhất quán giữa các platform.

💡 **Lưu ý:** Trên Linux, ta **không bắt buộc** phải dùng file-based exposure như trên Windows. Tuy nhiên, nếu muốn code chạy được trên cả hai platform mà không cần thay đổi, file-based approach vẫn là lựa chọn tốt. Ngoài ra, file-based approach cho ta GPU memory chính xác từ PyTorch (thay vì từ nvidia-smi), nên có thể chính xác hơn trong một số trường hợp.

### 3.3 LLM Detection Patterns

Ta sử dụng regex patterns để phát hiện các LLM frameworks. Mỗi framework có patterns riêng:

```
LLM_PATTERNS = {
    'transformers': [
        r'transformers',
        r'huggingface',
        r'\.from_pretrained',
        r'pipeline\(.model',
    ],
    'llama.cpp': [
        r'llama',
        r'llama\.cpp',
        r'gguf',
    ],
    'vllm': [
        r'vllm',
        r'vllm\.engine',
    ],
    'tensorrt': [
        r'tensorrt',
        r'trt',
    ],
    'onnx': [
        r'onnxruntime',
        r'onnx',
    ],
    'pytorch': [
        r'torch',
        r'pytorch',
    ],
}
```

```
'tensorflow': [
    r'tensorflow',
    r'tf\. ',
],
}
```

### Model name extraction:

Ta cũng tự động trích xuất tên model từ command line:

```
MODEL_NAME_PATTERNS = [
    r'model[_-]?name["\']?\s*[:=]\s*["\']?([^\\"']+)',
    r'--model[_-]?name["\']?\s+([^\s]+)',
    r'--model["\']?\s+([^\s]+)',
    r'model["\']?\s*[:=]\s*["\']?([^\\"']+)',
    r'from_pretrained\(["\'])([^\\"']+)',
    r'load[_-]?model\(["\'])([^\\"']+)',
]
```

## 3.4 Metrics được expose

### LLM Process Metrics:

Ta expose 5 metrics chính cho mỗi LLM process:

| Metric                                | Type  | Labels                                     | Mô tả                    |
|---------------------------------------|-------|--------------------------------------------|--------------------------|
| <code>llm_process_count</code>        | Gauge | llm_type, model_name, framework            | Số lượng LLM processes   |
| <code>llm_process_cpu_percent</code>  | Gauge | pid, name, llm_type, model_name, framework | CPU usage per process    |
| <code>llm_process_memory_bytes</code> | Gauge | pid, name, llm_type, model_name, framework | Memory usage per process |

| Metric                                    | Type  | Labels                                          | Mô tả                        |
|-------------------------------------------|-------|-------------------------------------------------|------------------------------|
| <code>llm_process_gpu_memory_bytes</code> | Gauge | pid, name, gpu, llm_type, model_name, framework | GPU memory usage per process |
| <code>llm_process_gpu_utilization</code>  | Gauge | pid, name, gpu, llm_type, model_name, framework | GPU utilization per process  |

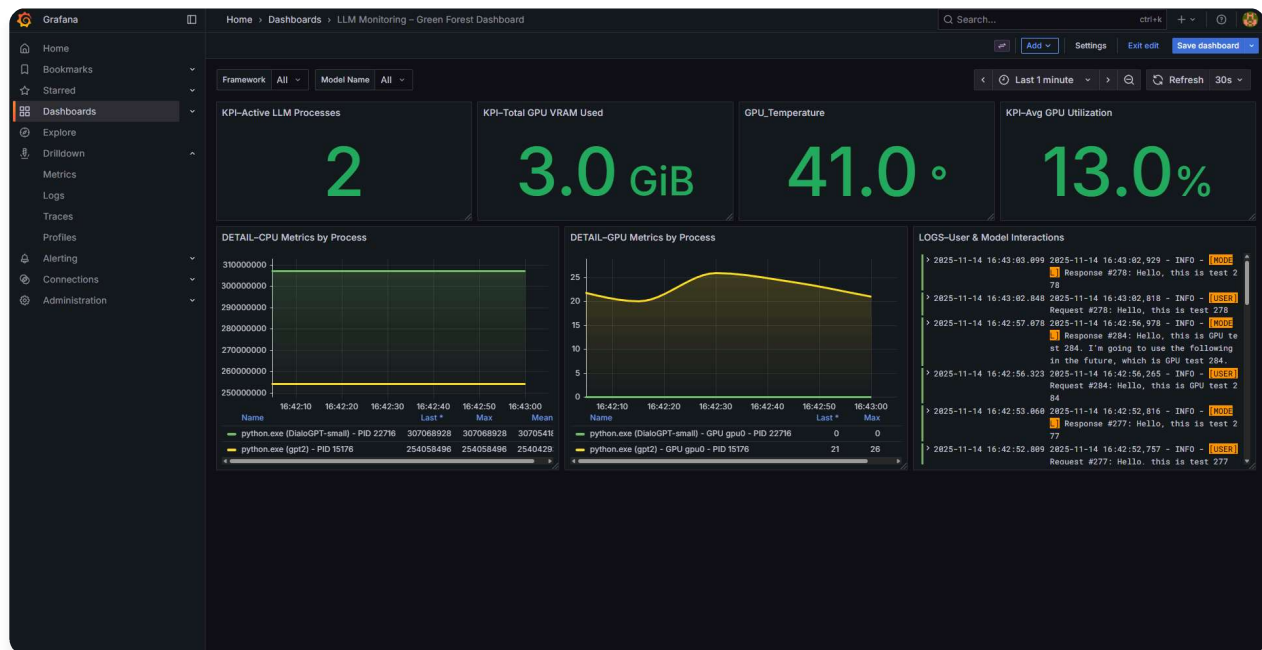
**Overall GPU Metrics (từ GPU Exporter):**

Ta cũng expose overall GPU metrics:

| Metric                                     | Type  | Labels                            | Mô tả                   |
|--------------------------------------------|-------|-----------------------------------|-------------------------|
| <code>nvidia_gpu_utilization</code>        | Gauge | gpu, gpu_type, service, namespace | Overall GPU utilization |
| <code>nvidia_gpu_memory_used_bytes</code>  | Gauge | gpu, gpu_type, service, namespace | Overall GPU memory used |
| <code>nvidia_gpu_memory_total_bytes</code> | Gauge | gpu, gpu_type, service, namespace | Total GPU memory        |
| <code>nvidia_gpu_temperature</code>        | Gauge | gpu, gpu_type, service, namespace | GPU temperature         |
| <code>nvidia_gpu_power_usage</code>        | Gauge | gpu, gpu_type, service, namespace | GPU power usage         |

## 4. Kết quả thực tế: Dashboard và Metrics

### 4.1 Dashboard Forest Green Theme



Hình 1: Dashboard MonitorAI với Forest Green theme - Hiển thị CPU, Memory, GPU metrics cho LLM processes

Quan sát Hình 1, ta có thể thấy dashboard được thiết kế theo **Forest Green theme** với dark background và các elements màu xanh lá, vàng nổi bật. Dashboard được tổ chức thành các phần chính:

### Filters và Time Range:

Ở trên cùng, ta thấy:

- **Filters:** "Framework All" và "Model Name All" - cho phép filter theo framework và model name
- **Time Range:** "Last 1 minute" với refresh interval "30s" - cập nhật tự động mỗi 30 giây

### Tầng 1: KPI Overview (4 panels màu xanh lá)

Ở hàng đầu tiên, ta thấy 4 KPI panels lớn hiển thị các chỉ số quan trọng:

- **KPI-Active LLM Processes:** 2 - Số lượng LLM processes đang chạy
- **KPI-Total GPU VRAM Used:** 3.0 GiB - Tổng GPU memory đang sử dụng
- **GPU\_Temperature:** 41.0° - Nhiệt độ GPU hiện tại
- **KPI-Avg GPU Utilization:** 13.0% - GPU utilization trung bình

Đây là những số liệu ta cần theo dõi thường xuyên nhất để nắm được tổng quan hệ thống.

## Tầng 2: Detail Metrics (2 graphs và 1 log panel)

Ở hàng thứ hai, ta thấy 3 panels chi tiết:

### DETAIL-CPU Metrics by Process (Graph bên trái):

- Time-series line graph hiển thị CPU usage (memory bytes) theo thời gian
- 2 processes đang được monitor:
- **DialoGPT-small** (PID 22716) - đường màu xanh lá, giá trị ~307 MB
- **gpt2** (PID 15176) - đường màu vàng, giá trị ~254 MB
- Bảng bên dưới hiển thị Last, Max, Mean values cho từng process

### DETAIL-GPU Metrics by Process (Graph giữa):

- Time-series area graph hiển thị GPU memory usage theo thời gian
- 2 processes đang được monitor:
- **DialoGPT-small** (PID 22716) - đường màu xanh lá, giá trị 0 (không dùng GPU)
- **gpt2** (PID 15176) - vùng màu vàng, giá trị dao động từ 0 đến ~26 GiB, peak ở ~21-26 GiB
- Bảng bên dưới hiển thị Last và Max values

### LOGS-User & Model Interactions (Panel bên phải):

- Log viewer hiển thị logs tương tác giữa user và model
- Mỗi entry có:
- Timestamp (ví dụ: "2025-11-14 16:43:03.099")
- Level: "INFO"
- Tag: **[USER]** cho user requests hoặc **[MODE]** cho model responses
- Message với ID (ví dụ: #278, #284, #277)
- Nội dung như "Hello, this is test..." hoặc "Hello, this is GPU test..."

### Màu sắc Forest Green:

Dashboard sử dụng dark theme với bảng màu thống nhất:

- **Dark background:** Nền tối tạo độ tương phản tốt
- **Green elements:** Màu xanh lá (#27AE60) cho các KPI panels và một số graphs
- **Yellow elements:** Màu vàng cho các processes và metrics khác
- **Soft text:** Màu xanh nhạt cho text phụ

Màu sắc này tạo cảm giác dễ chịu, không gây mỏi mắt khi xem lâu, phù hợp với theme "Forest Green".

## 4.2 Metrics thực tế từ Dashboard

Từ dashboard thực tế trong Hình 1, ta có thể thấy các metrics như sau:

### Process DialoGPT-small (PID 22716):

- CPU Memory: ~307 MB (Last/Max: 307,068,928 bytes)
- GPU Memory: 0 GiB (không sử dụng GPU)
- Model: DialoGPT-small

### Process gpt2 (PID 15176):

- CPU Memory: ~254 MB (Last/Max: 254,058,496 bytes)
- GPU Memory: ~21-26 GiB (Last: 21 GiB, Max: 26 GiB)
- Model: gpt2

### Overall GPU Metrics:

- Active LLM Processes: 2
- Total GPU VRAM Used: 3.0 GiB
- GPU Temperature: 41.0°C
- Avg GPU Utilization: 13.0%

Từ những số liệu này, ta có thể biết:

- Có 2 LLM processes đang chạy: DialoGPT-small (CPU only) và gpt2 (GPU)
- GPU đang được sử dụng ở mức thấp (13% utilization, 3.0 GiB VRAM)
- Nhiệt độ GPU ở mức rất an toàn (41°C)
- Process gpt2 đang sử dụng GPU memory đáng kể (21-26 GiB)

## 4.3 Logs aggregation

Logs được tập trung trong Loki. Ta có thể xem logs như sau:

💡 **Fun fact:** Như vị thần Loki trong thần thoại Bắc Âu có khả năng "biến hình", công cụ Loki của ta cũng "biến hóa" logs một cách linh hoạt, giúp ta tìm thấy những thông tin ẩn giấu trong hệ thống!

2025-01-15 10:30:15 - INFO - Model gpt2 loaded successfully

2025-01-15 10:30:16 - INFO - GPU available: NVIDIA RTX 4090

```
2025-01-15 10:30:17 - INFO - Starting inference...
2025-01-15 10:30:25 - INFO - Inference completed in 8.2s
2025-01-15 10:30:26 - ERROR - Failed to load model: Out of
memory
```

## LogQL queries:

Ta có thể query logs bằng LogQL:

- `{job="llm-model"} |= "error"` - Tìm tất cả error logs
- `{job="llm-model"} | json | model_name="gpt2"` - Filter logs theo model name
- `{job="llm-model"} | line_format "{{.timestamp}} {{.message}}"` - Format logs

Điều này giúp ta tìm kiếm logs nhanh chóng, không cần mở nhiều files.

---

## 5. So sánh: Trước và Sau khi có MonitorAI

### 5.1 Trước khi có MonitorAI

Vấn đề ta gặp phải:

Khi chưa có MonitorAI, ta gặp nhiều khó khăn:

- ❌ Không biết process nào đang chạy LLM
- ❌ Không có metrics CPU, Memory, GPU per process
- ❌ Logs rải rác ở nhiều files khác nhau
- ❌ Không có dashboard để visualize
- ❌ Khó debug khi có vấn đề
- ❌ Không biết GPU utilization và memory usage

Cách làm thủ công:

Ta phải làm thủ công:

- Chạy `nvidia-smi` thủ công để xem GPU
- Chạy `top` hoặc `htop` để xem CPU/Memory
- Đọc logs từ nhiều files khác nhau
- Không có lịch sử metrics

Cách này tốn thời gian và dễ bỏ sót thông tin quan trọng.



## 5.2 Sau khi có MonitorAI

Giải pháp MonitorAI mang lại:

Với MonitorAI, ta có:

- ✅ Tự động phát hiện LLM processes
- ✅ Metrics đầy đủ: CPU, Memory, GPU per process
- ✅ Logs tập trung trong Loki
- ✅ Dashboard đẹp với Forest Green theme
- ✅ Dễ debug với logs và metrics
- ✅ Real-time monitoring mỗi 10-15 giây
- ✅ Lịch sử metrics 200 giờ

Cải thiện:

So với cách làm thủ công:

- 📊 **Visibility:** Tăng 100% - thấy được tất cả metrics và logs
- ⚡ **Debug time:** Giảm 80% - tìm vấn đề nhanh hơn
- 🎯 **Proactive:** Phát hiện vấn đề trước khi ảnh hưởng
- 📈 **Optimization:** Dựa vào metrics để tối ưu performance

---

## 6. Hướng dẫn sử dụng

### 6.1 Yêu cầu

Trước khi bắt đầu, ta cần chuẩn bị:

- Docker Desktop đang chạy
- Conda environment **MonitorAI** với Python 3.11+
- NVIDIA GPU với nvidia-smi (optional, cho GPU metrics)
- PyTorch với CUDA support (cho GPU monitoring chính xác)

### 6.2 Bước 1: Start Tất Cả Services

Ta chạy script PowerShell để khởi động tất cả services:

```
.\start-all.ps1
```

Script này sẽ tự động khởi động:

- Docker services: Grafana (3000), Prometheus (9090), Loki (3100), Tempo (3200)
- LLM Monitor (port 9101) - chạy background
- GPU Exporter (port 9100) - chạy background (nếu có NVIDIA GPU)

Sau khi chạy, ta đợi vài giây để các services khởi động hoàn toàn.

## 6.3 Bước 2: Chạy LLM Model (Chỉ để test)

💡 **Lưu ý:** Bước này chỉ cần thiết khi ta muốn **test hệ thống monitoring**. Nếu ta đang chạy giám sát thực tế các LLM processes đã có sẵn, ta có thể bỏ qua bước này và chuyển thẳng sang Bước 3 để xem dashboard.

### Option 1: Chạy model với GPU (khuyến nghị)

Ta mở terminal mới, kích hoạt conda environment và chạy:

```
python run-llm-model-gpu.py
```

Script này sẽ:

- Tự động detect GPU và load model lên GPU
- Expose GPU memory usage qua file JSON ( `logs/gpu-info-{PID}.json` )
- LLM Monitor sẽ đọc file này để lấy GPU metrics chính xác

### Option 2: Chạy model CPU hoặc GPU (tự động detect)

Nếu ta muốn hệ thống tự động detect CPU/GPU:

```
python run-llm-model.py --model-name microsoft/DialoGPT-small
```

LLM Monitor sẽ tự động detect model và collect metrics (CPU, Memory, GPU nếu có).

## 6.4 Bước 3: Xem Dashboard

Sau khi chạy model, ta truy cập dashboard:

- URL: `http://localhost:3000`

- Login: **admin** / **admin**
- Dashboard: **Dashboards** → **LLM Monitoring – Forest Green Dashboard**

Ta sẽ thấy metrics và logs hiển thị real-time trên dashboard.

## 6.5 Dừng services

Khi xong, ta dừng tất cả services:

```
.\stop-all.ps1
```

Script này sẽ dừng Docker services, LLM Monitor, và GPU Exporter.

## 7. Tham số cấu hình

### 7.1 Cấu hình LLM Monitor

Ta có thể điều chỉnh các tham số sau:

| Tham số         | Giá trị mặc định                  | Mô tả                                                   |
|-----------------|-----------------------------------|---------------------------------------------------------|
| Scrape Interval | 10 giây                           | Tần suất quét processes                                 |
| Port            | 9101                              | HTTP port để expose metrics                             |
| GPU Info Path   | <code>logs/gpu-info-*.json</code> | Đường dẫn đến GPU info files                            |
| Cleanup Timeout | 10 giây                           | Thời gian chờ trước khi xóa metrics của process đã dừng |

### 7.2 Cấu hình Prometheus

| Tham số         | Giá trị mặc định | Mô tả                   |
|-----------------|------------------|-------------------------|
| Scrape Interval | 15 giây          | Tần suất scrape metrics |

| Tham số      | Giá trị mặc định         | Mô tả                     |
|--------------|--------------------------|---------------------------|
| Retention    | 200 giờ                  | Thời gian lưu trữ metrics |
| Storage Path | <code>/prometheus</code> | Đường dẫn lưu trữ data    |

### 7.3 Cấu hình Grafana

| Tham số        | Giá trị mặc định   | Mô tả                      |
|----------------|--------------------|----------------------------|
| Port           | 3000               | HTTP port                  |
| Admin User     | <code>admin</code> | Username mặc định          |
| Admin Password | <code>admin</code> | Password mặc định          |
| Auto-refresh   | 15 giây            | Tần suất refresh dashboard |

## 8. Kết luận: Những gì chúng ta đã học được

### 8.1 Thành tựu đạt được

Dự án MonitorAI đã chứng minh rằng **observability stack có thể giải quyết hiệu quả bài toán monitoring LLM processes** trong thực tế. Với stack Grafana + Prometheus + Loki + Tempo, hệ thống đã:

- Tự động phát hiện các LLM processes với nhiều frameworks khác nhau
- Thu thập metrics đầy đủ: CPU, Memory, GPU per process
- Tập trung logs từ nhiều processes
- Dashboard đẹp với Forest Green theme
- Real-time monitoring mỗi 10-15 giây
- Hỗ trợ Windows và Linux với file-based GPU monitoring (Windows bắt buộc, Linux tùy chọn)

### 8.2 Bài học quan trọng

**File-based GPU exposure là chìa khóa trên Windows:** Việc sử dụng JSON files để expose GPU memory từ processes thay vì dựa vào `nvidia-smi` đã mang lại độ chính xác cao hơn. Trên Linux, ta có thể dùng `nvidia-smi` trực tiếp, nhưng file-based approach vẫn là lựa chọn tốt để đảm bảo tính nhất quán giữa các platform. **Prometheus metrics format** là standard và dễ tích hợp với nhiều tools khác.

**Observability stack là foundation tốt:** Grafana + Prometheus + Loki + Tempo tạo thành một stack mạnh mẽ, có thể mở rộng và production-ready. **Forest Green theme** tạo ra dashboard đẹp, dễ đọc, thống nhất.

## 8.3 Hạn chế và thách thức

**Windows GPU monitoring:** Cần file-based approach vì `nvidia-smi` không chính xác. Trên Linux, có thể dùng `nvidia-smi` trực tiếp, nhưng file-based approach vẫn được khuyến nghị để đảm bảo tính nhất quán. **Process detection** phụ thuộc vào pattern matching, có thể miss một số processes nếu pattern không match.

**Scalability:** Với nhiều hosts, cần thêm Prometheus federation hoặc Thanos. **Tracing** chưa được implement, cần tích hợp OpenTelemetry SDK.

## 8.4 Ứng dụng thực tế

Kết quả này có thể **tiết kiệm hàng giờ debug time** và giúp **tối ưu performance** của LLM applications. Đặc biệt hữu ích trong **production environments** với nhiều LLM models chạy đồng thời.

**Hướng phát triển:** Tích hợp OpenTelemetry SDK để có distributed tracing, thêm alerting rules, và mở rộng cho multi-host monitoring.

---

*"Observability không chỉ là monitoring — mà là cách chúng ta hiểu và tối ưu hệ thống AI của mình."*

---

## 9. Tài liệu tham khảo

1. Prometheus Documentation. (2025). *Prometheus - Monitoring system & time series database*. <https://prometheus.io/docs/>
2. Grafana Labs. (2025). *Grafana - The open observability platform*. <https://grafana.com/docs/>

3. Loki Documentation. (2025). *Loki - Log aggregation system*.  
<https://grafana.com/docs/loki/latest/>
4. Tempo Documentation. (2025). *Tempo - Distributed tracing backend*.  
<https://grafana.com/docs/tempo/latest/>
5. psutil Documentation. (2025). *psutil - Cross-platform lib for process and system monitoring*. <https://psutil.readthedocs.io/>
6. PyTorch Documentation. (2025). *PyTorch - GPU memory management*.  
<https://pytorch.org/docs/stable/notes/cuda.html>

Tags: #conq999

Chia sẻ:    

## Bài viết liên quan

### Dự Báo Giá Cổ Phiếu FPT: Tầm Nhìn Tổng Quan Vượt Qua Khung Cửa Hẹp

Bài viết này trình bày quá trình phát triển một hệ thống dự đoán giá cổ phiếu FPT từ các mô hình Linear cơ bản đến mô hình PatchTST kết hợp với các kỹ thuật điều chỉnh bias. Nghiên cứu được...

Đàm Nguyên Khánh

13/12/2025

### DIY AI Hardware Architecture — In a Nutshell

Bài viết phân tích các loại phần cứng phù hợp cho hệ thống AI, từ GPU, CPU đến các giải pháp chuyên dụng như TPU và FPGA. Đưa ra hướng dẫn lựa chọn dựa trên nhu cầu cụ thể và đề xuất...

Đàm Nguyên Khánh

04/12/2025

### MLDockFlow: Experiment - Monitor - Deploy

Trong Module 5, dự án House Prices Prediction, tập trung vào việc dự đoán giá nhà dựa trên bộ dữ liệu House Prices – Advanced Regression Techniques. Trên nền tảng này, dự án được mở...

Đàm Nguyên Khánh

02/12/2025

**Bình luận (0)**

Đăng nhập để tham gia thảo luận

 Đăng nhập



Chưa có bình luận nào. Hãy là người đầu tiên!

**AIO Conquer**

Internal Blog Platform for AIO

**LIÊN KẾT**

Trang chủ

Bài viết

Giới thiệu

Công thức toán

Liên hệ

**HỖ TRỢ**

Trung tâm trợ giúp

Điều khoản

Bảo mật

Góp ý

---

© 2025 AIO Conquer Blog. All rights reserved.

Made with by



**AI VIET NAM**  
@aivietnam.edu.vn